CUDA/OpenCL

TERMINOLOGIJA

	CUDA	OpenCL
CPU	host	host
GPU	device	compute device
GPU jezgra	SM	compute unit
dretva	thread	work item
blok dretvi	thread block	work group
skup blokova	grid	NDRange/index range
val dretvi	warp	wavefront
dijeljena memorija	shared memory	local memory
privatna memorija	private memory	private memory

MEMORIJA

	CUDA	OpenCL
globalna memorija	__global__	__global
dijeljena memorija bloka/grupe	__shared__	__local
privatna memorija dretve	__private__ (*default*)	__private (*default*)

SINKRONIZACIJA

CUDA: `__syncthreads()`

OpenCL: `barrier()`

INDEKSIRANJE

1D		
	CUDA	**OpenCL**
lokalni indeks	threadIdx.x	get_local_id(0)
indeks bloka/grupe	blockIdx.x	get_group_id(0)
veličina bloka/grupe	blockDim.x	get_local_size(0)
broj blokova/grupa	gridDim.x	get_num_groups(0)
globalni indeks	blockIdx.x * blockDim.x + threadIdx.x	get_group_id(0) * get_local_size(0) + get_local_id(0)

2D		
	CUDA	**OpenCL**
lokalni x	threadIdx.x	get_local_id(0)
lokalni y	threadIdx.y	get_local_id(1)
blok/grupa x	blockIdx.x	get_group_id(0)
blok/grupa y	blockIdx.y	get_group_id(1)
veličina x	blockDim.x	get_local_size(0)
veličina y	blockDim.y	get_local_size(1)
br. blokova x	gridDim.x	get_num_groups(0)
br. blokova y	gridDim.y	get_num_groups(1)
globalni stupac	blockIdx.x * blockDim.x + threadIdx.x	get_global_id(0)
globalni redak	blockIdx.y * blockDim.y + threadIdx.y	get_global_id(1)
pristup matrici	data[row * width + col]	data[row * width + col]

PRIMJER KERNELA

CUDA

```
__global__ void VecAdd(float* A,float* B,float* C){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    C[i] = A[i] + B[i];
}
```

OPENCL

```
__kernel void VecAdd(__global float* A,__global float* B, __global float* C){
    int i = get_global_id(0);
    C[i] = A[i] + B[i];
}
```